Epsilon Greedy Search for Open-ended Text Generation

Ghassen Tlig

Open-ended text generation (Open-TG) is a pivotal task in natural language processing (NLP) with significant applications in service-oriented computing, facilitating tasks such as story genera-tion, dialogue systems, and content creation. Traditional methods like beam search often struggle to balance exploiting known good paths and exploring new possibilities, leading to repetitive or less creative output.

We introduce an Epsilon Greedy Search approach for open-ended text generation that employs a probabilistic mechanism to alternate between exploiting high-probability sequences and exploring less probable but potentially richer continuations. The proposed Epsilon Greedy Search method addresses this gap by enhancing the flexibility and responsiveness of text-based services, support-ing more robust interactions in applications such as automated customer support, personalized recommendations, and dynamic content delivery.

1 Introduction

Language modeling, such as Word2Vec, BERT, and GPT-3, aims to learn a probability distribution, $p_\theta(x)$, for a variable-length text sequence $x = \{x_1, \ldots, x_n\}$, where θ denotes model parameters. Many methods have been proposed to compute this probability distribution, allowing the language model to learn a mapping function between an unlabeled text dataset and probability estimation. Typically, maximum likelihood estimation (MLE) is used to train the language model.

There has been significant progress in language modeling techniques, evolving from very naive n-gram models to the most advanced models such as GPT-4. These models typically begin with pre-training on large datasets, followed by fine-tuning for specific tasks, resulting in end-to-end solutions with minimal human intervention. The Transformer model, introduced in [12], has played a pivotal role in this evolution, harnessing increased computational power to facilitate the development of pre-trained language models such as BERT [1] and OpenAI GPT [9]. These models, often referred to as Large Language Models (LLMs), can encompass tens to hundreds of billions of parameters. Transformers serve as the fundamental architecture for these models, employing encoder-decoder structures. During the decoding process, these models generate token-by-token text, predicting each subsequent token based on previously generated tokens. The decoder components translate encoded sequences and token prediction vectors into coherent text, leveraging embeddings from the encoder and decoding methods to predict the next word in the output text.

Despite their success, LLMs still face challenges such as hallucinations, repetitive text, and incoherence, which are actively addressed in the subfield of Open-ended Text Generation (Open-TG). Open-TG aims to generate fluent continuations of given prompts, defined as follows:

Open-ended Text Generation (Open-TG):

- Input: Given a prompt prefix, $X_{pre} = [x_1, \ldots, x_n]$ where x_i, $i = 1, \ldots, n$ is a token in the vocabulary V ,
- Objective: Generate a fluent and coherent continuation of X_{pre} of length m denoted as $X_{cont} = [x_{n+1}, \ldots, x_{n+m}]$

Open-TG plays a crucial role in various NLP tasks such as text completion [9], automated story generation [2], question-answering systems [4], and automatic text summarization [6,13]. In practical applications, text generation improves automatic content creation, chatbot responses, email marketing, and social media content creation for businesses.

We propose a decoding procedure based on a greedy search approach designed to generate natural text for Open-TG problems. Our decoding algorithm incorporates both deterministic and stochastic elements. The primary intuition behind epsilon greedy search is to combine deterministic and stochastic approaches within the same decoding method.

2 Related work

The decoding procedure aims to generate an output sequence of tokens, one at a time, conditioning each token on the previously generated ones. This approach is used in various natural language processing tasks, including text generation and text completion. Fundamentally, the decoder algo-rithm utilizes the probability distribution from a pre-trained language model to predict the next token, ensuring that the prediction for a token at position idepends only on the output tokens at positions preceding. The main approaches in this area include contrastive decoding and contrastive search.

2.1 Contrastive decoding

Contrastive decoding, proposed by Li et al. [5], utilizes two language models: a large LM and a small LM. This approach aims to optimize the disparity between the probability distributions generated by these models. The contrastive decoding objective is formulated as an optimization problem and can be expressed as follows:

$$\max_{X_{cont}} \{ \log p_{EXP}(X_{cont}|X_{pre}) - \log p_{AMA}(X_{cont}|X_{pre}) \} \tag{1}$$

The objective function of contrastive decoding (CD) aims to introduce a discrepancy between patterns generated by an amateur LM and those produced by an expert LM. Consequently, this function penalizes the amateur model for correct predictions and rewards expert patterns, even when they seem implausible. The primary challenge of contrastive decoding lies in balancing the high confidence scores given by an expert LM against the low scores assigned by an amateur LM, where the small LM is frequently incorrect, and the expert LM occasionally produces surreal outputs. To mitigate this issue, a plausibility constraint is incorporated to augment the CD objective model and address these weaknesses. Thus, the revised CD model is formulated as follows:

$$\begin{aligned} &\max_{X_{cont}} \{\log p_{EXP}(X_{cont}|X_{pre}) - \log p_{AMA}(X_{cont}|X_{pre})\} \\ &\text{s.t.} \quad x_i \in V_{head}(x_{<i}), \forall x_i \in X_{cont} \end{aligned} \tag{2}$$

where $V_{head}(x_{<i}) = \{x_i \in V : p_{EXP}(w|x_{<i}) \geq \alpha \max_w p_{EXP}(w|x_{<i})\}$, is an adaptive plausibility condition that defines the acceptance of the next token. It takes priority of tokens favored by the large LM and dismiss the undesired tokens that was generated by small LM (see Fig. 1).

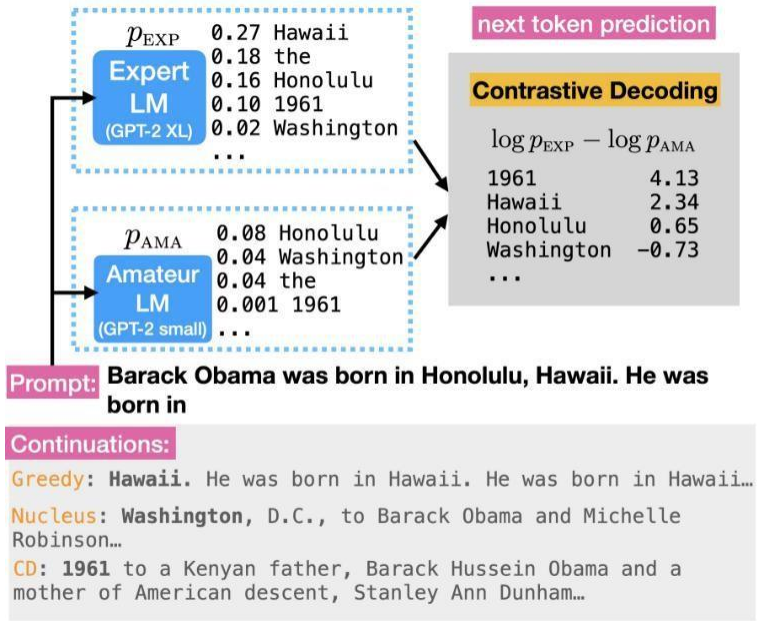

Fig. 1. A prompt example of contrastive decoding using two language models LM: a large language model (LLM, i.e. GPT-E XL) and a small language model (SLM, i.e. GPT-2 small), (source [5])

2.2 Contrastive search.

Yixuan et al. [11,10] introduced a text generation framework that integrates a training phase called contrastive training with a decoding method known as contrastive search, aimed at generating completed texts:

– This phase introduces a new language model estimation, denoted as LSimCT G, which combines maximum likelihood estimation (LM LE) with contrastive learning (LCL). Contrastive learning aims to minimize discrepancies between different tokens to mitigate repetitive loops and inco-herent text during decoding. The new training objective is defined as $L_{SimCT\ G} = L_{MLE} + L_{CL}$, where L_{MLE} is the maximum likelihood estimation and L_{CL} defines the contrastive training estimation as following:

$$L_{CL} = \frac{1}{|x|(|x|-1)} \sum_{i=1}^{|x|} \sum_{j=1:j=i}^{|x|} \max\left(0, \rho - s(h_{x_i}, h_{x_i}) + s(h_{x_i}, h_{x_j})\right) \tag{3}$$

– During each decoding step, the framework selects the output token with the highest probability while minimizing redundancy with the previous context. A penalty is applied to discourage undesirable redundancies in the generated text. At each step t, the selection of the output x_t follows:

$$x_t = \underset{v \in V^{(k)}}{\operatorname{argmax}} \ (1-\alpha)p_\theta(v|x_{<t}) - \alpha(\max\{s(h_v, h_{x_j} : 1 \le j \le t-1\}) \tag{4}$$

3 Epsilon greedy search

We present an epsilon greedy decoding approach capable of generating text continuations by com-bining the deterministic and stochastic aspects of the decoding methods, as depicted in Algorithm 1.

At each step, a hyperparameter ε is initialized to determine which aspect to employ during text generation. When ε is set to a high value, the epsilon greedy prioritizes selecting the text continuation with the highest probability according to the maximum likelihood model's distribution. Alternatively, when ε is lower, epsilon greedy expands the selection to consider a set of top-k potential tokens for the next step. Each token is assigned a probability score, which is then smoothed using interpolation to enhance coherence and select the most probable and coherent continuations. Formally, given the previous context $x_{<t}$, at time step t, the selection of the output x_t follows:

$$x_t = \underset{v \in V^{(k)}}{\text{random}} \; p_\theta(v|x_{<t}) : \text{where } V^{(k)} \text{ is the set of top-k predictions} \tag{5}$$

The hyperparameter $\varepsilon \in [0, 1]$ determines the importance of these two aspects of decoding. When $\varepsilon = 1$, epsilon greedy search degenerates into greedy algorithm.

Algorithm 1: Epsilon Greedy Search for Open-ended Text Generation

Input:
– $p_\theta(x)$ a probability distribution for a text vocabulary V , and $X_{pre} = [x_1, \ldots, x_n]$ a prompt prefix

Output: $X_{cont} = [x_{n+1}, \ldots, x_{n+m}]$ a fluent and coherent continuation of X_{pre}
Parameters: $\varepsilon \leftarrow\!- \{0.5, 0.6, 0.7, 0.8\}$ and $K \leftarrow\!- \{4, 6, , 8, 10\}$

```
begin
    X ←− X_pre;
    repeat
        eps ←− random([0, 1]); ps
if ϵ < ε then
            x_t ←− argmax_{v∈V} {p_θ(v|x_<t)}; else
            begin
                Returns the K largest elemen
                L ←− max_{v∈V (K)} {p_θ(v|x_<t)};        ts of p_θ(x|x_<t);
                x_t ←− random_{v∈L}{p˜_θ(v|x_<t)};
        X.append(x_t);
                                                    /* –» X = [x_1, . . . , x_n, x_n+1, . . . , x_n+t] */
    until (m tokens generated);
```

4 Experiments

In automatic evaluation for text generation tasks, the focus is primarily on three key metrics: quality, diversity, and coherence [14].

We report the results of traditional decoding methods: greedy search, top-k sampling (with k = 50), nucleus sampling (with p = 0.95) [3] and typical sampling (with τ = 0.95) [7].

For all results (Tabs. 1, 2, 3), we compare all decoding methods under GPT2-XL model [9]

4.1 Automatic evaluation

Following previous studies , we present the results of the quality of text generation based on the following metrics:

- Diversity takes into account the generation repetition at different n-gram levels, and it is defined as:
  - $\text{div} = \prod_{n=2}^{4} (1.0 - \frac{\text{rep}-n}{100})$, where $\text{rep}-n = 100(1.0 - \frac{|\text{unique n-gram}(\hat{x})|}{|\text{total n-grams}(\hat{x})|}$
- MAUVE is designed for measuring the token distribution closeness between the generated text and the human-written text over the whole test set. Note that, while the maximum length of generation in the experiments is 256 (or 128),
- Coherence measures the semantic coherence between the prompt and the generated text. Formally, given the prompt x and the generated text ($\hat{x}$), coherence is defined as the averaged log-likelihood of ($\hat{x}$) conditioned on (x) as:
  - $\text{coh}(\hat{x}, x) = \frac{1}{|\hat{x}|} \sum_{i=1}^{|\hat{x}|} \log p_M(\hat{x}_i|[x : \hat{x}_{<i}])$, where [:] is the concatenation operation and (M) is a LLM.

Table 1. Automatic evaluation results for news domain generation (Wikinews dataset source: http://www.wikinews.org).

Method	% div ↑	MAUVE(% ↑)	coh ↑
Greedy search	3.55	13.96	-0.47
Top-k sampling	91.56	89.86	-2.22
Nucleus sampling	93.54	89.45	-2.61
Typical sampling	95.37	90.97	-3.26
Epsilon Greedy search	86,00	2.65	-0.33

Table 2. Automatic evaluation results for text generation (Wikitext dataset : source [8]).

Method	% div ↑	MAUVE(% ↑)	coh ↑
Greedy search	1.77	4.19	-0.41
Top-k sampling	87.49	81.00	-2.37
Nucleus sampling	92.16	86.54	-3.03
Typical sampling	94.82	86.07	-3.71
Epsilon Greedy search	1.77	4.91	-0.40

Table 3. Automatic evaluation results for story domain generation.

Method	% div ↑	MAUVE(% ↑)	coh ↑
Greedy search	0.86	2.56	-0.34
Top-k sampling	91.22	87.49	-2.45
Nucleus sampling	94.50	91.47	-3.02
Typical sampling	96.29	88.58	-3.68
Epsilon Greedy search	3.55	13.96	-0.46

References

1. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (eds.) Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (Jun 2019). https://doi.org/10.18653/v1/N19-1423, https://aclanthology.org/N19-1423
2. Fan, A., Lewis, M., Dauphin, Y.: Hierarchical neural story generation. In: Gurevych, I., Miyao, Y. (eds.) Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 889–898. Association for Computational Linguistics, Melbourne, Australia (Jul 2018). https://doi.org/10.18653/v1/P18-1082, https://aclanthology.org/P18-1082
3. Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y.: The curious case of neural text degeneration (2020), https://arxiv.org/abs/1904.09751
4. Li, W., Duan, M., An, D., Shao, Y.: Large language models understand layouts (2024), https://arxiv.org/abs/2407.05750
5. Li, X.L., Holtzman, A., Fried, D., Liang, P., Eisner, J., Hashimoto, T., Zettlemoyer, L., Lewis, M.: Contrastive decoding: Open-ended text generation as optimization. In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) Proceedings of the 61st Annual Meeting of the Association for Computa-tional Linguistics (Volume 1: Long Papers). pp. 12286–12312. Association for Computational Lin-guistics, Toronto, Canada (Jul 2023). https://doi.org/10.18653/v1/2023.acl-long.687, https://aclanthology.org/2023.acl-long.687
6. Liu, Y., Lapata, M.: Text summarization with pretrained encoders. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). pp. 3730–3740. Association for Computational Linguistics, Hong Kong, China (Nov 2019). https://doi.org/10.18653/v1/D19-1387, https://aclanthology.org/D19-1387
7. Meister, C., Pimentel, T., Wiher, G., Cotterell, R.: Locally typical sampling. Transactions of the Association for Computational Linguistics 11, 102–121 (2022), https://api.semanticscholar.org/CorpusID:252918666
8. Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. ArXiv abs/1609.07843 (2016), https://api.semanticscholar.org/CorpusID:16299141
9. Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners (2019), https://api.semanticscholar.org/CorpusID:160025533
10. Su, Y., Collier, N.: Contrastive search is what you need for neural text generation (2023), https://arxiv.org/abs/2210.14140
11. Su, Y., Lan, T., Wang, Y., Yogatama, D., Kong, L., Collier, N.: A contrastive framework for neu-ral text generation. ArXiv abs/2202.06417 (2022), https://api.semanticscholar.org/CorpusID: 246823043
12. Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polo-sukhin, I.: Attention is all you need. In: Neural Information Processing Systems (2017), https://api.semanticscholar.org/CorpusID:13756489
13. Zhang, H., Cai, J., Xu, J., Wang, J.: Pretraining-based natural language generation for text summarization. In: Bansal, M., Villavicencio, A. (eds.) Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL). pp. 789–797. Association for Computational Linguistics, Hong Kong, China (Nov 2019). https://doi.org/10.18653/v1/K19-1074, https://aclanthology.org/K19-1074
14. Zhu, Y., Kiros, R., Zemel, R.S., Salakhutdinov, R., Urtasun, R., Torralba, A., Fidler, S.: Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. 2015 IEEE International Conference on Computer Vision (ICCV) pp. 19–27 (2015), https://api.semanticscholar.org/CorpusID:6866988